

Algoritmos e Estruturas de Dados II

Roteiro de Laboratório – Árvore AVL

Objetivo

Implementar uma Árvore AVL a partir de uma Árvore Binária de Pesquisa (ABP).

Pré-requisitos

- Código funcional de uma Árvore Binária de Pesquisa (classes No e ArvoreBinaria).
- Compreensão dos conceitos de balanceamento em árvores binárias a partir de rotações simples e duplas

Atividades

1. Adicionar atributo nível

Alterar a classe No para incluir um novo atributo chamado “nível” do tipo inteiro. Este atributo irá armazenar a quantidade de níveis que existem abaixo do nó (incluindo o próprio nó). Veja o exemplo a seguir que apresenta os valores de níveis para os nós em uma AVL:



2. Criar método para retornar o nível de um nó

Criar um método na classe No “private static int getNivel(No i)” que retorna o nível do nó i passado como parâmetro. Atenção: o nível de um nó inexistente é 0.

3. Criar método para atualizar o nível do nó

Criar um método na classe No “public void setNivel()” que atualiza o nível do nó, seguindo a definição fornecida para o atributo. Dica: pense na regra que define o nível de um nó como uma equação:

$$\text{nível} = 1 + \text{MAX}(\text{nível}(\text{filho à esquerda}), \text{nível}(\text{filho à direita}))$$

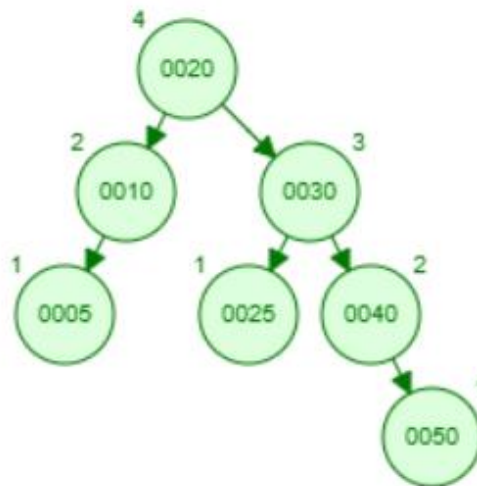
4. Atualizar os níveis dos nós na inserção

Alterar o método de inserção recursivo da classe ArvoreBinaria para que, antes de retornar o ponteiro para o nó, o nível deste nó seja atualizado.

5. Validar o cálculo dos níveis

Criar um programa para validar que os níveis dos nós estão sendo calculados com sucesso. Para isso, seu programa deve:

- Criar uma árvore vazia.
- Inserir os elementos na árvore conforme exemplo a seguir.
- Caminhar na árvore e, em cada visita, imprimir o elemento visita juntamente com seu nível.



Para esta árvore, espera-se que seu programa imprima no caminhamento central:

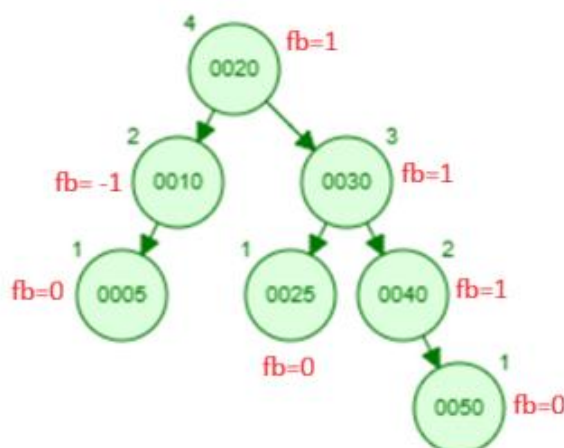
5(1) 10(2) 20(4) 25(1) 30(3) 40(2) 50(1)

6. Criar método para obter fator de balanceamento

Criar um método na classe No “public int getFatorBalanceamento()” que calcula e retorna o fator de balanceamento do nó. O fator de balanceamento do nó é calculado com base nos níveis dos filhos, seguindo a regra:

fator de balanceamento = nível(filho à direita) - nível(filho à esquerda)

Veja o exemplo a seguir onde são apresentados, para cada nó da AVL, o nível e o fator de balanceamento.



7. Validar o cálculo dos níveis

Alterar seu programa para que, em cada visita, também seja impresso o fator de balanceamento do nó. Para essa árvore, espera-se que seu programa imprima no caminhamento central:

5(1)(fb=0) 10(2)(fb=-1) 20(4)(fb=1) 25(1)(fb=0) 30(3)(fb=1) 40(2)(fb=1) 50(1)(fb=0)

8. Criar os métodos para os casos de balanceamento

Criar, na classe `ArvoreBinaria`, os métodos para os casos de balanceamento (veja nossos slides!):

- Rotação simples à esquerda
- Rotação simples à direita
- Rotação dupla direita-esquerda
- Rotação dupla esquerda-direita

Cada método deve receber um ponteiro para o nó a ser rotacionado, aplicar a rotação e retornar um ponteiro para o nó que é a nova raiz do conjunto após a rotação. Lembre-se de atualizar o nível dos nós que sofreram alteração de nível no processo de rotação.

9. Criar o método para balancear um nó

Criar um método na classe `ArvoreBinaria` “private No balancear(No i)” que recebe um ponteiro para um nó e aplica as regras de balanceamento neste nó (veja nossos slides!). O método deve retornar um ponteiro para o nó que é a nova raiz do conjunto após o balanceamento. Lembrem-se das regras de balanceamento da AVL:

Fator do nó	Fator do filho à direita	Fator do filho à esquerda	Tipo de rotação
2	1 0		Simple à esquerda
	-1		Dupla dir-esq
-2		-1 0	Simple à direita
		1	Dupla esq-dir

10. Integrar o balanceamento à inserção

Alterar o método de inserção recursivo da classe `ArvoreBinaria` para que, antes de retornar o ponteiro para o nó, aplique o balanceamento neste nó e retorne o ponteiro para o nó que é raiz resultante do balanceamento.

11. Validar a AVL

Alterar seu programa para que insira elementos na AVL e verifique se os balanceamentos estão sendo aplicados corretamente. Use o exemplo a seguir para verificação.

Ordem de inserção: 4, 35, 10, 13, 3, 30, 15, 12, 7, 40 e 20

